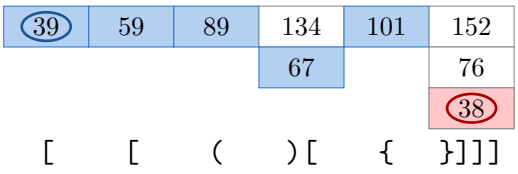
Collatz, but stop at the drop

A game with two letters: the proofs

Ruud H.G. van Tol and **Mike Winkler**
with assistance from Claude (Anthropic) and ChatGPT (OpenAI)

October 2026



© 2026 Ruud H.G. van Tol and Mike Winkler. Licensed under CC BY 4.0.

1 Only go all the way down if needed

Take a positive integer x. If it is even, halve it. If it is odd, triple it, add 1, and halve the (now even) result. In one formula, the *step* T:

$$T(x) = \begin{cases} x/2, & x \text{ even}, \\ (3x+1)/2, & x \text{ odd}, \end{cases}$$

the reduced Collatz function, OEIS A014682 [13]. Starting from 7, repeated steps give

$$7 \to 11 \to 17 \to 26 \to 13 \to 20 \to 10 \to 5.$$

The well-known question is whether every start eventually reaches 1. We ask something smaller: does every $x > 1$ eventually fall *below itself*? For 7 that happens after 7 steps, at 5. We stop right there, at the *drop*. In this paper we use the convention that $\mathrm{drop}(1) = 1$, with trajectory $(1, 2, 1)$. For every $x > 1$, reaching 1 is already a drop. The two questions are equivalent: if every number drops, then every number reaches 1, because each drop lands on a smaller number, which drops in turn (Lean: tDrop_equiv_collatzT for T; drop_equiv_collatz for the fast map R of Chapter 6; rDrop_iff_tDrop carries the drop question from one to the other). From here on we only ever look at the way to the drop, never beyond it.

Families that drop together

Take the odd numbers of the form $4i + 1$: $1, 5, 9, 13, \ldots$. One step gives

$$\frac{3(4i+1)+1}{2} = \frac{12i+4}{2} = 6i + 2,$$

which is even, so the next step halves it to $3i + 1$. (The number of factors 2 in $6i + 2$ is A375961(i) [31].) So after two steps, every number $4i + 1$ has become $3i + 1$, which is smaller as soon as $i \geq 1$. (Whether $3i + 1$ is odd or even depends on i; we don't need to know, because we have already dropped.) The whole *family* $1, 5, 9, 13, \ldots$ drops in two steps: for $i = 0$ the two steps are $1 \to 2 \to 1$, the drop of 1 by our convention. We write this as

$$1 + i \cdot 2^2 \;\to\; 1 + i \cdot 3^1.$$

The family $16i + 3$ takes four steps: $16i + 3 \to 24i + 5 \to 36i + 8 \to 18i + 4 \to 9i + 2$, that is,

$$3 + i \cdot 2^4 \;\to\; 2 + i \cdot 3^2.$$

The mechanism is visible in the i-part. A halving divides it by 2; an odd step multiplies it by 3/2. After p_2 steps, p_3 of them odd, the i-part $i \cdot 2^{p_2}$ has become $i \cdot 3^{p_3}$. As long as there are enough factors 2 left in the i-part, it doesn't influence whether a number is odd or even: every member of the family takes the same steps as its smallest member. Here are the first families, sorted by p_3 (Table 1).

family		after the drop	steps
$1 + i{\cdot}2^2$	$\to$	$1 + i{\cdot}3^1$	1 0
$3 + i{\cdot}2^4$	$\to$	$2 + i{\cdot}3^2$	1 1 0 0
$11 + i{\cdot}2^5$	$\to$	$10 + i{\cdot}3^3$	1 1 0 1 0
$23 + i{\cdot}2^5$	$\to$	$20 + i{\cdot}3^3$	1 1 1 0 0
$7 + i{\cdot}2^7$	$\to$	$5 + i{\cdot}3^4$	1 1 1 0 1 0 0
$15 + i{\cdot}2^7$	$\to$	$10 + i{\cdot}3^4$	1 1 1 1 0 0 0
$59 + i{\cdot}2^7$	$\to$	$38 + i{\cdot}3^4$	1 1 0 1 1 0 0

Table 1: The first families: class id (Chapter 3) plus multiples of 2^{p_2}, the values after the drop, and the steps.

In the last column, 1 is an odd step and 0 a halving. Every family drops, because $3^{p_3} < 2^{p_2}$ and the constant part drops too. And in this table the number of steps is the same for a given p_3: it is the number of binary digits of 3^{p_3}. (That it is so for every family is the width identity of Chapter 7.) For example, $3^4 = 81 = 1010001_2$ has 7 digits.

Which *step patterns* are possible, and how many are there for each p_3? Chapter 2 answers the first question exactly; the counts from $p_3 = 1$ on, $1, 1, 2, 3, 7, 12, 30, 85, \ldots$, form OEIS A100982 [19]. With this the question becomes: *does every $x > 1$ belong to some family?* That is still open, and Chapter 7 splits it into two precise parts. What this paper shows is how the families are built, how to count them, and how they sort:

- A member of a family drops at the end of its pattern exactly when it lies above a single rational number, its *cycle point* (Chapter 3).
- Every step pattern splits at its record lows into smaller patterns, and whether pieces fit together depends only on their p_3 (Chapter 4).
- The words are counted in three ways, one of them extended by M. Winkler (A100982, Chapter 5).
- Families, their step patterns, and the patterns of a faster map sort in the same order (Chapter 6).

How this paper relates to Paper 1

Paper 1 [39] tells the same story as a game, without proofs. This paper gives the proofs, in the words of the literature and of the Lean formalization: Paper 1's valid words are the admissible words here, its height is the level, its Collatz-primes are the class ids, and its parts are the blocks. Every result cited with a Lean name is proved in Lean 4 with Mathlib (open statements enter as hypotheses); Lean names are set in sans-serif, like admissible_iff_chain (namespace AcceleratedCollatz). Examples use the script `pv.py` of Paper 1, Appendix A.

Paper 1 reads its words off the real rides and stays shallow on purpose. What it leaves out is here:

- the words defined by levels instead of rides, and every rule of Paper 1 proved for them (Chapter 2); whether levels and rides always agree is the width identity, and together with covering it is the open part (Chapter 7);
- the drop formula and the cycle point (Chapter 3);
- the exact gluing rule, the block factor and its records, brackets and other ways to cut (Chapter 4);
- the counting formulas, first passages, active words and Winkler's bounds (Chapter 5);
- marked vectors and the sorting proofs (Chapter 6);
- the finite check of the width identity up to Order 10 781 273, and five side results (Chapter 7).

Contributions

M. Winkler pointed to the classical ladder factorization behind the cuts, gave the range form of the binomial identity and the deficit form of the budget, and asked the question that led to the sweeps for every odd q.

Key terms introduced in Chapter 1:	
step T	$x \mapsto x/2$ for even x, $x \mapsto (3x+1)/2$ for odd x.
drop	the first value below the start, and the step where it is reached.
family	all $x = c + i \cdot 2^{p_2}$ that take the same steps up to their drop; later called a *class*, with *class id* c.
p_3 (Order)	the number of odd steps up to the drop.
p_2	the number of binary digits of 3^{p_3}: the length of the word (Chapter 2) and, under the width identity, the number of steps up to the drop (Chapter 7).
step pattern	the odd steps (1) and halvings (0) up to the drop; later called an *admissible word*.
A014682	the step T in the OEIS.
A375961	factors 2 in $6i + 2$.

2 Words and levels

Chapter 1 showed that inside a family only the step pattern matters. Let us study the patterns on their own, without numbers.

A *word* is a finite string of 1s and 0s. Read it from left to right and keep a running product, the *level*: start at 1, multiply by 3/2 for every 1 and by 1/2 for every 0. This is what happens to the i-part of a family (Chapter 1), so the level tells how a family's members grow and shrink, ignoring the small "+1" of each odd step. After a word with n ones and length L, the level is $3^n/2^L$.

Table 2 shows the word 1101100, the pattern of $7 + i \cdot 2^7$.

letter	1	1	1	0	1	0	0
level	$\frac{3}{2}$	$\frac{9}{4}$	$\frac{27}{8}$	$\frac{27}{16}$	$\frac{81}{32}$	$\frac{81}{64}$	$\frac{81}{128}$

Table 2: The levels of the word 1101100, letter by letter.

The level stays above 1 until the very last letter, and then falls below 1. That is the shape asked of a pattern: its level must not fall below 1 too early, and it must do so at the end.

A word is *admissible* if its level is above 1 after every proper, nonempty beginning (prefix), and below 1 after the whole word (Lean: Admissible). Its *Order* is its number of 1s. Admissible words are *first passages*: the first moment the level goes below 1.

Three facts follow directly, and together they explain Table 1.

- *An admissible word ends with 0* (admissible_last_false). A final 1 would raise the level from at least 1 to at least 3/2.
- *An admissible word of Order n has length $p_2(n)$, the number of binary digits of 3^n* (admissible_length). Just before the final 0 the level $3^n/2^{L-1}$ is at least 1, and after it $3^n/2^L$ is below 1, so $2^{L-1} \le 3^n < 2^L$. Equality holds only for the word 0: a power of 2 is never a power of 3, except $1 = 1$. These lengths are $1, 2, 4, 5, 7, 8, 10, \ldots$ (A020914 [14]); from one Order to the next they grow by 1 or 2 (A022921 [15]).
- *No admissible word is a proper prefix of another* (admissible_not_prefix). The shorter one already brings the level below 1, which the longer one forbids.

The second fact says that the order of the letters doesn't change the length: all admissible words of Order n are rearrangements of n ones and $p_2(n) - n$ zeros. Here are all seven of Order 5:

11011010, 11011100, 11101010, 11101100, 11110010, 11110100, 11111000.

They are listed in *lexicographic* (dictionary) order, which is also the order in which `pv.py` numbers them: this is their *rank*.

The 0s at the end of an admissible word carry no information: the Order fixes the length. Read as a binary number, the word without them is the *odd part* of the word, its number divided by 2 as often as possible (A000265 [11]; encode_core, Lean: core). For Order at least 1 it ends with the last 1, and filled up with 0s to the length p_2 it gives the word back (keyPV_core, core_keyPV). So distinct admissible words, of any Orders, have distinct odd parts (oddPart_injective). `pv.py` takes and prints odd parts (`pv.py list 5` begins with `1101101`) and computes all it prints from them; attrs does so in Lean.

Counting without listing

The level after a prefix depends only on how many 1s and 0s it has, not on their order. So a prefix is a point (j, k) in a grid, j ones and k zeros, and an admissible word is a walk from $(0, 0)$ that, after its first step, stays in the region $3^j > 2^{j+k}$ until its last step leaves it. Counting walks is quick: add up the counts of the two predecessors of each point. This gives the number $a(n)$ of admissible words of Order n (A100982, $n \geq 1$) without listing a single word; `pv.py count 60` prints $a(60)$, a 25-digit number, instantly. The counts per grid point form the triangle A368514 [29], similar to the Catalan triangle but with $p_2(n) - n$ entries in row n; the last nonzero entry of its row $n + 1$ is $a(n)$.

The same walk, counted by length instead of Order, gives two more sequences. A word whose level stays above 1 all the way is *active*; there are $1, 1, 1, 2, 3, 4, 8, 13, 19, \ldots$ of length $k = 0, 1, 2, \ldots$ (A076227 [18]). An active word followed by 1 is active again; followed by 0 it is either active or admissible. So the admissible words of length k number $2\,\mathrm{A076227}(k-1) - \mathrm{A076227}(k)$; that count is nonzero only at the lengths $k = p_2(n)$, where it equals $a(n)$ (for $n \geq 1$, and as far as the width identity of Chapter 7 holds, this is A260591 [25], which counts by dropping time). So $a(n) = 2\,\mathrm{A076227}(p_2(n) - 1) - \mathrm{A076227}(p_2(n))$ (card_active_succ, card_admSet_add_active).

The word at a given rank

The grid also gives the word at any rank without listing the others, as the ballot numbers do for Dyck paths. Fix the Order n and count backwards: let $t(j, k)$ be the number of admissible words of Order n that begin with a given prefix at the point (j, k). It is 1 at the end point $(n,\, p_2(n) - n)$, and inside the region

$$t(j, k) = t(j, k + 1) + t(j + 1, k),$$

where a term counts as 0 if its point has more than n ones, or lies outside the region and is not the end point. Then $a(n) = t(1, 0)$. Table 3 shows t for $n = 5$, with the end point at $(5, 3)$.

$k\backslash j$	1	2	3	4	5
0	7	7	5	3	1
1	·	2	2	2	1
2	·	·	·	1	1
3	·	·	·	·	1

Table 3: The backward count $t(j, k)$ for Order 5: j ones across, k zeros down.

To write the word of rank r, start with 1 at $(1, 0)$. At the point (j, k) let $z = t(j, k + 1)$, the number of words that go on with 0. If $r \leq z$, write 0; otherwise write 1 and replace r by $r - z$. For $n = 5$ and $r = 4$ the values of z are $0, 2, 2, 0, 1, 1, 1$ and the word is 11101100 (`pv.py unrank 5 4`). The other way, the rank of a word is 1 plus the sum of z over its 1s after the first (`pv.py rank`).

The table has no more than $n\,p_2(n)$ entries, and a word takes $p_2(n)$ steps. For Order 60, with $a(60) = 132543803671227413053631 4$, the word of rank $\lfloor 2\,a(60)/3 \rfloor = 883625357808182753690876$ is

```
111111010001111111111011010100111010111101101101101
001011101011000101111101101011111100001100010100
```

with the class id 56358534677734342530833494207. At Order 2000, where $a(2000)$ has 902 digits, this still takes less than a second. The table, the walk and its inverse are also in Lean, with ranks counted from 0: the table gives $a(n)$ (card_admSet_eq_count), and the walk and its inverse compute the sorted enumeration of the words of an Order and its inverse (admIndex_eq_unrank, admIndex_symm_eq_rank).

A warning about names

Terras [36] used *admissible* for a weaker condition: level above 1 after every proper prefix, with no condition at the end. In his words, our admissible words are *terminal*, and words whose level also stays above 1 at the end are *active*. We follow the usage of OEIS A100982. The two meet in one number: $a(n)$ is the entry of Terras's table for length $p_2(n)$ with $p_2(n) - n$ zeros, since no word of that length and Order is active.

Key terms introduced in Chapter 2:	
word	a string of 1s (odd steps) and 0s (halvings).
level	the running product: $\times\frac{3}{2}$ per 1, $\times\frac{1}{2}$ per 0; after the whole word it is $3^n/2^L$.
admissible word	level above 1 after every proper nonempty prefix, below 1 at the end; a first passage. Terras: *terminal*.
first passage	the first prefix whose level is below 1.
Order	the number of 1s in a word.
$\boldsymbol{p_2(n)}$	the length of every admissible word of Order n: the number of binary digits of 3^n.
$\boldsymbol{a(n)}$	the number of admissible words of Order n (A100982).
lexicographic order	dictionary order on words of equal length.
rank	position of a word in the lexicographic list of its Order.
odd part	a word without the 0s at its end; as a binary number, its odd part (A000265).
active word	level above 1 after every nonempty prefix, including the whole word.
A368514	active prefixes counted by ones and zeros.
A020914, A022921	$p_2(n)$, and its increments 1 or 2.
A100982	$a(n)$, admissible words of Order n.
A076227, A260591	active, admissible words of length k (A260591 as far as the width identity holds).

3 From words to numbers: the cycle point

Now bring the numbers back. The *parity word* of x of length L records its first L steps: 1 for an odd step, 0 for a halving.

Every word is a class

Which numbers have a given parity word w of length L? Chapter 1 already showed the mechanism: the first step is decided by the last binary digit of x, the second step by the last two digits, and so on. Each step fixes one more binary digit. So the numbers with parity word w form exactly one residue *class* modulo 2^L:

$$x = c + i \cdot 2^L, \qquad i = 0, 1, 2, \ldots,$$

where $1 \le c \le 2^L$ is its smallest positive member, the *class id* (Lean: tBits_classId, tBits_congr, modEq_of_tBits_eq; for the admissible words of one Order, classEquiv, classId_tBits). For admissible words we call the class ids *Collatz-primes*. Grouped by Order, they are the rows of A177789 [21]:

$$2 \mid 1 \mid 3 \mid 11, 23 \mid 7, 15, 59 \mid 39, 79, 95, 123, 175, 199, 219 \mid \ldots$$

(A177789 writes the Order-0 class as $0 \bmod 2$). Merged and sorted from Order 2 on, they form A243115 [24]: $3, 7, 11, 15, 23, 27, 31, 39, 47, 59, \ldots$. The smallest of each Order, $2, 1, 3, 11, 7, 39, \ldots$, is A122442 [20] (which has 5 in the place of the 1: the smallest member above 1). These three sequences are defined by dropping times; they are our class ids as far as the width identity of Chapter 7 holds.

The drop formula

Each odd step adds a 1 before halving. Following these +1s through the remaining steps shows that for every x whose first p_2 steps follow an admissible word w of Order p_3,

$$T^{p_2}(x) = \frac{3^{p_3}\, x + \rho}{2^{p_2}},$$

where ρ is an integer that depends only on w, positive when the Order is at least 1 (Lean: rho, rho_pos_iff). For $w = 11010$, class id 11: $\rho = 23$, and indeed $(27 \cdot 11 + 23)/32 = 320/32 = 10$. Replacing x by $x + i \cdot 2^{p_2}$ adds exactly $i \cdot 3^{p_3}$ to the result: that is the family rule $c + i \cdot 2^{p_2} \to d + i \cdot 3^{p_3}$ of Chapter 1 (tStep_iterate_shift; in terms of the drop, for a class id above its cycle point, affine_classId).

Since the result is an integer, $3^{p_3} c + \rho$ is divisible by 2^{p_2}. As 3 is invertible modulo 2^{p_2}, this finds the class id directly:

$$c \equiv -\rho \cdot 3^{-p_3} \pmod{2^{p_2}}.$$

For 11010: $27^{-1} \equiv 19$ and $-23 \cdot 19 \equiv 11 \pmod{32}$. This is how `pv.py` computes class ids.

Who drops: above the cycle point

A member x drops at the end exactly when $3^{p_3}x + \rho < 2^{p_2}x$, that is, when

$$\rho < d_{23}\, x, \qquad d_{23} = 2^{p_2} - 3^{p_3},$$

(drop_iff_rho). And no member drops *before* the end, because every proper prefix has level above 1 (no_drop_before). Write $x^* = \rho/d_{23}$. Then:

a member of the class drops at the end of its word if and only if it is larger than x^*.

The number x^* is the *cycle point* of the word: an integer $x = x^*$ that plays the word would come back to itself (fixed_iff_rho). Table 4 shows some examples.

class id	word	ρ	d_{23}	x^*
1	1 0	1	1	1
3	1 1 0 0	5	7	5/7
11	1 1 0 1 0	23	5	23/5
7	1 1 1 0 1 0 0	73	47	73/47
39	1 1 1 0 1 1 0 0	251	13	251/13

Table 4: Cycle points $x^* = \rho/d_{23}$ of the first words.

The first line explains why 1 drops only onto itself: it *is* its own cycle point, with trajectory $(1, 2, 1)$. Every other member of that class, $5, 9, 13, \ldots$, lies above it and drops.

The cycle point carries more. Its denominator d_{23} is odd and not divisible by 3, so x^* can be read modulo powers of 2 and of 3. Read modulo 2^{p_2}, it *is* the class id; read modulo 3^{p_3}, it is T^{p_2} of the class id (its drop, if it drops there). For $x^* = 23/5$: $23 \cdot 5^{-1} \equiv 11 \pmod{32}$ and $23 \cdot 5^{-1} \equiv 10 \pmod{27}$. One rational number, three readings: its 2-adic digits give the start, its 3-adic digits the drop, and its size decides whether the drop happens.

How close does a class id come to its cycle point? The ratio $q = x^*/x$ must be below 1 for the class id to drop. Among all admissible words of Orders 2 to 15 the largest ratio is $q = 251/507$, at $x = 39$ (`pv.py maxq 2 ... 15`). Among the class ids below 20 000 000 the tightest case is $x = 47$ (Order 34), whose cycle point lies between 33 and 34. For 27, whose way to the drop takes 59 steps, the cycle point lies between 8 and 9: far below. Whether *every* class id above 1 lies above its cycle point is open; it is the width identity of Chapter 7, checked there for every Order up to 10 781 273.

All residues modulo 2^k together, with the step T, form a De Bruijn graph (Laarhoven and de Weger [7]); the admissible classes are a sparse, special part of it.

Key terms introduced in Chapter 3:

parity word of x	its first L steps: 1 odd, 0 halving.
class	all x with the same parity word of length L; a residue class modulo 2^L.
class id	the smallest positive member of a class.
Collatz-prime	a class id of an admissible word.
A177789, A243115	class ids by Order; all of Order ≥ 2, sorted (as far as the width identity holds).
A122442	the smallest class id of each Order (as far as the width identity holds).
ρ	the integer with $T^{p_2}(x) = (3^{p_3}x + \rho)/2^{p_2}$.
d_{23}	$2^{p_2} - 3^{p_3}$, positive for admissible words.
cycle point x^*	ρ/d_{23}; members above it drop at the end, none drop earlier.
q	x^*/x; the class id drops when $q < 1$.

4 The chain decomposition

This chapter is the heart of the paper: a way to cut any admissible word into smaller admissible words, and to glue them back.

Cutting at record lows

Take an admissible word of Order at least 1. It starts with 1, so the level begins at 3/2. Now cut the rest each time the level reaches a new *record low*, that is, each time the piece since the last cut first brings its own product below 1. For the word 11011010 (class id 123):

$$1 \mid 10 \mid 11010, \qquad \text{levels at the cuts: } \tfrac{3}{2} \to \tfrac{9}{8} \to \tfrac{243}{256}.$$

Each piece is an admissible word in its own right: 10 has Order 1 and 11010 has Order 3. We call such pieces *blocks*. The cut positions are exactly the moments where the level is lower than ever before, counted from the leading 1 on (isRecordLow_iff_cut, exists_chain_recordLow). Cutting a path at these *descending ladder epochs* is a classical idea [3], as M. Winkler pointed out; what is new here is what it gives for admissible words: the condition below, which depends on the Orders only, and the counts of Chapter 5.

Theorem 1 (Chain decomposition)**.** *Every admissible word of Order $n+1$ is, in exactly one way, a* chain

$$1\, A_1 A_2 \cdots A_k\, t,$$

with $k \geq 0$ admissible blocks $A_1, \ldots, A_k$ of Orders $o_1, \ldots, o_k \geq 1$, $o_1 + \cdots + o_k = n$, and a tail *t that is empty or a single 0. Conversely, admissible blocks of any such Orders glue together in this way into an admissible word if and only if the level at the cut before the last block is above 1:*

$$2^{\,1+p_2(o_1)+\cdots+p_2(o_{k-1})} \;<\; 3^{\,1+o_1+\cdots+o_{k-1}},$$

and then the tail is forced: it is 0 exactly when the inequality also holds with o_k included.

Lean: exists_chain, chain_unique, admissible_iff_chain, validOrders_iff_dropLast; the tail in terms of p_2, tail_eq_zero_iff.

Why it works. Cutting at the record lows gives the blocks: each piece is the shortest one with product below 1, so it is admissible, and a last piece 0 is the tail. Inside a block, the level is the level at the block's start times a prefix product of the block, and both are above 1. So the only places where the level can fail are the block boundaries, and there the level is $3^{1+o_1+\cdots+o_i}/2^{1+p_2(o_1)+\cdots+p_2(o_i)}$, which depends only on the Orders. Each block lowers the level, so it suffices to check the last inner cut. Uniqueness comes from the prefix fact of Chapter 2.

The surprise is the second half: *which* blocks are used doesn't matter at all, only their Orders. The seven words of Order 5 show it (`pv.py list 5`, Table 5).

class id	word	chain
123	11011010	1 | 10 | 11010
219	11011100	1 | 10 | 11100
199	11101010	1 | 11010 | 10
39	11101100	1 | 1101100
79	11110010	1 | 11100 | 10
175	11110100	1 | 1110100
95	11111000	1 | 1111000

Table 5: The seven admissible words of Order 5 and their chains.

The Orders $(1,3)$, $(3,1)$ and (4) appear, each with every choice of blocks: $1 \cdot 2$, $2 \cdot 1$ and 3 words. The other five, such as $(2,2)$ or $(1,1,2)$, never appear: they pass level 1 too early.

Chains as codes

Theorem 1 gives a bijection: on one side the admissible words of Order $n+1$, on the other the lists of blocks with *valid Orders* (Orders that satisfy its inequality) that add up to n (chainEquiv). A block is an admissible word itself, known by its Order and its rank (Chapter 2). A word can be written as a short list of pairs (Order, rank), one per block: its *code*. The leading 1 is not listed, and the tail follows from the Orders. This is a bijection as well (codeEquiv). For 11011010 the blocks 10 and 11010 are the first words of Orders 1 and 3, so its code is $(1,1),(3,1)$; 11101100 has the single block 11101100, the first word of Order 4, and the code $(4,1)$. Both maps are computed by one pass that takes, again and again, the shortest piece with product below 1 (chainEquiv_eq_chainOf, codeEquiv_eq_codeOf). In Lean the ranks count from 0; `pv.py` counts from 1 and lists a tail as $(0,1)$ (`pv.py attrs 11`, for 1100).

The odd part of a word answers a what-if. Stop an odd start $x > 1$ that drops at the end of its admissible word, not at its drop but as soon as the odd part of the value is below x. From the last odd step on only halvings remain, down to the drop value $d < x$; there the odd part of the value is that of d, which is below x. Earlier it is not: a value $2^k m$ with m odd and below x would halve straight down to m, with no odd step left. This stop comes right after the last odd step: its word is the odd part, and the odd part of its value is that of the drop value. Classes, Orders and counts stay as they are. But the odd parts of one Order differ in length, and one can begin another (1 begins 11), so the three facts of Chapter 2 are lost. We keep the drop.

The block factor

A block of Order o multiplies the level by its *block factor* $f(o) = 3^o/2^{p_2(o)}$, which lies between $1/2$ and 1: for example $f(1) = 3/4$, $f(3) = 27/32$ and $f(5) = 243/256$. Starting from $3/2$, the level must stay above 1 at the last inner cut. This is a *budget*: the product of the block factors of all blocks but the last must stay above $2/3$. For the chain 1 | 10 | 11010 it is $f(1) = 3/4 > 2/3$ (Figure 2). M. Winkler phrased the same budget with logarithms: the deficits $1 - \{o \log_2 3\}$ of those blocks must add up to less than $\log_2 3 - 1$. So long chains need blocks with $f(o)$ close to 1. The Orders where $f(o)$ sets a new record are

$$1,\ 3,\ 5,\ 17,\ 29,\ 41,\ 94,\ 147,\ 200,\ 253,\ 306,\ \ldots$$

(Figure 1; `pv.py records 306`). These are the denominators of the upper convergents and semiconvergents of $\log_2 3$ (upperRecord_iff_semiconv): the fractions on the Stern–Brocot path towards a number are its simplest upper and lower approximations, and the runs of the path are the quotients of Euclid's algorithm [4, §4.5, pp. 122–123]; M. Winkler [42] links them to the admissible words. Together with the lower ones (lowerRecord_iff_semiconv) they form A206788 [23]. Walking down the Stern–Brocot tree of fractions towards $\log_2 3$ (K. Ryde [35]), these record Orders are exactly the denominators of the fractions on the walk that lie above $\log_2 3$ (upperRecord_iff_sbWalk). In terms of p_2 alone: $p_2(a+b)$ is $p_2(a)+p_2(b)$ or one less. At a record Order n it is one less for every split $n = a + b$ into positive parts (upperRecord_iff_add). At the Orders where f comes closer to $1/2$ than ever before, the *lower* record Orders $1, 2, 7, 12, 53, \ldots$, it is the full sum for every split (lowerRecord_iff_add), and they are the denominators of the fractions below $\log_2 3$ on that walk (lowerRecord_iff_sbWalk); the record Orders above are the *upper* ones. The first word of each Order (rank 1) takes every halving that keeps the level above 1 (unrank_zero). After a of its odd steps and their halvings (not the final one) its level is $2f(a)$, and every other level before its end is above 2. So its chain is cut exactly there for the lower record Orders a up to its own Order (isRecordLow_firstWord): its blocks are the first words of the Orders $1, 5, 5, 41, 306, 306, \ldots$, the gaps between neighbouring lower record Orders, and of the rest (firstWord_chain; `pv.py first 53`). Each such gap is an upper record Order (upperRecord_sub_of_lowerRecord). Taken again for the rest, the first word of Order n is the first words of $a_1, a_2, \ldots$, each without its final halving, then one halving, where a_1 is the largest lower record Order up to n, a_2 the largest up to $n - a_1$, and so on (for $n = 30$: $12, 12, 2, 2, 2$; firstWord_eq_greedy).

Brackets

Open a bracket at every odd step of a word, and close it where the level first falls below the level just before that odd step. For each odd step, count the brackets that close before the next odd step; this list of counts is the *bracket word*. It is enough to rebuild the word: two admissible words with the same bracket word are equal (bracketWord_injective). The reason is short. For every level ℓ above 1 and up to the current level, some open bracket was opened at a level of at most ℓ and above $\ell/2$ (lvl_gap), so every halving closes at least one bracket (closesAt_pos_of_halving); at the first place where two words differ, the one with the halving closes more brackets before the next odd step. For odd $q \geq 5$ (the sweeps below) an odd step opens e brackets, e the largest with $2^e \leq q$, at $1, 2, \ldots, 2^{e-1}$ times that level; then too the word is rebuilt (bracketWordQ_injective), with one bracket not (for $q = 5$: 10100, 11000).

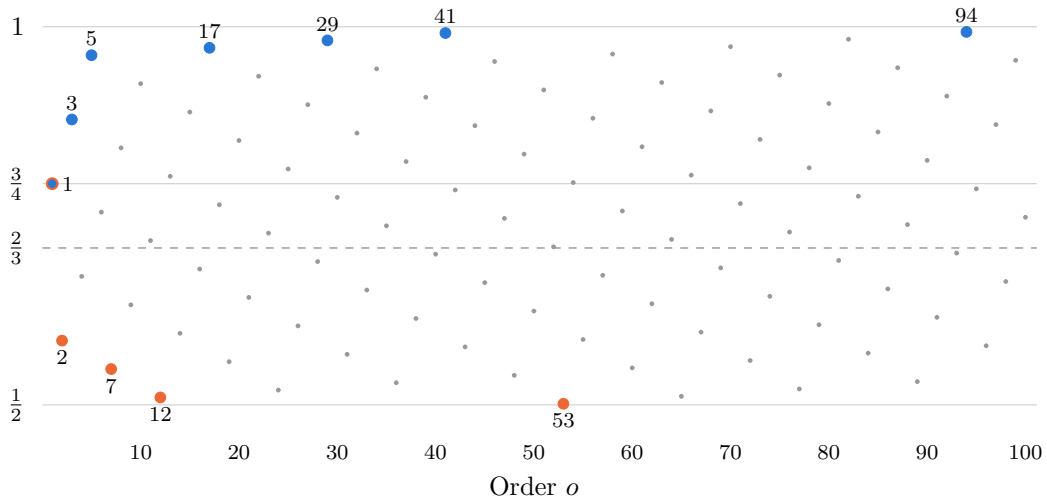


Figure 1: The block factor $f(o) = 3^o/2^{p_2(o)}$ for the Orders 1 to 100, on a logarithmic scale between 1 and 1/2. Blue: the Orders where $f(o)$ comes closer to 1 than ever before, the record Orders of the text; orange: those where it comes closer to 1/2 (1 is both). Dashed: the budget 2/3. A block above the dashed line can stand first in a chain of two or more blocks; one below it can only be the last block.

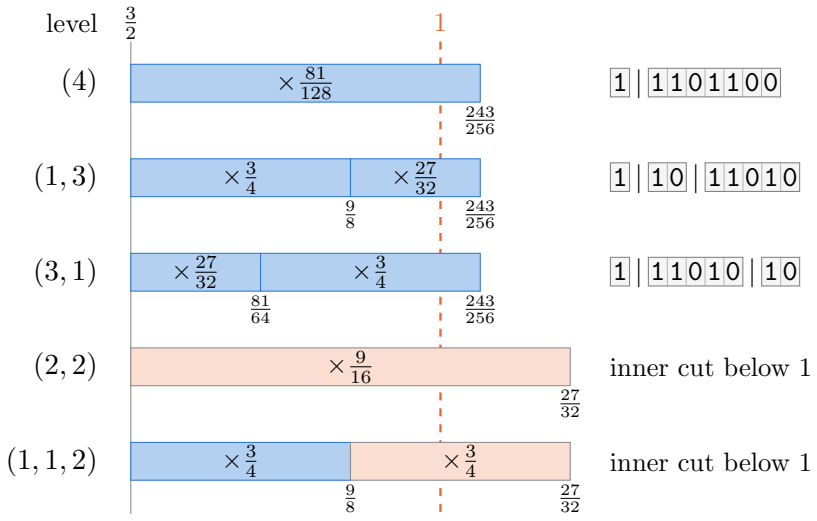


Figure 2: The budget of a chain, for Order 5, with the level on a logarithmic scale. Every chain starts at level 3/2, after the leading 1, and each block multiplies the level by its block factor; the level at each cut is written below it. Every block but the last must end above level 1 (dashed). The Orders (4), (1, 3) and (3, 1) fit, each shown with one of its words; all three end at 243/256, so their tail is empty. With (2, 2) and (1, 1, 2) an inner cut falls below 1.

On a trajectory brackets can be read off the values too: open at every odd value, close at its first drop (Figure 3). A value cannot fall below its bracket's start before the level does (exists_lvlDrop_le_of_isFirstDrop). With the width identity at the start of a bracket, both close at the same step (lvlDrop_iff_isFirstDrop): under it for every start, every level close in an admissible word is the drop of the bracket's start, with drop(1) = 1 (lvlDrop_iff_isTDrop). For 39 both give the bracket line of Figure 3; they agree for every odd start up to 200 000, at each of the 596 245 floors on its way to the drop, in words of at most 135 letters (an exact computation outside Lean).

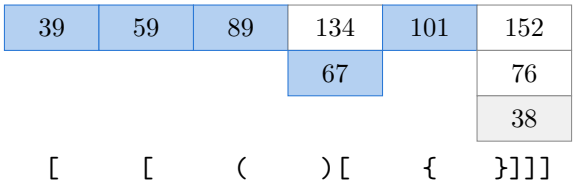


Figure 3: The drop grid of 39: the start, then one column per odd step, with the value it gives and the halvings after it below; odd values are shaded blue; 39 drops to 38 after 8 steps. Under each column, the brackets that open and close there. Every bracket opens at an odd value and closes where the value first falls below it: with) at an odd value, with } at an even value, with] at the final drop.

In the grid of Chapter 2, the walk of a word of Order $n \geq 1$ ends at $(n, p_2(n) - n)$, which is $2n - p_2(n)$ steps short of the diagonal. Every bracket closes, and only after a halving, so the bracket word is a Dyck path with n up and n down steps (sum_bracketWord, sum_colCount_lt_le). The down steps are the halvings of the word and $2n - p_2(n)$ *shared closes*: where one halving closes several brackets at once, every close after the first (halvings_le_colCount, sum_sharedCloses). Winkler's rational Dyck paths at the record Orders [42] use the halvings alone; the bracket word adds where the shared closes fall.

Which Dyck paths are bracket words? Theorem 1 answers it. In the chain $1\,A_1 \cdots A_k\,t$ every bracket of a block closes inside that block, and the bracket of the leading $\boxed{1}$ closes at the very end. So the bracket word of the chain is a 0 followed by the bracket words of $A_1, \ldots, A_k$, with 1 added to the last entry (bracketWord_chain); the word $\boxed{1}\boxed{0}$ has the bracket word (1). For example $\boxed{1}\,|\,\boxed{1}\boxed{0}\,|\,\boxed{1}\boxed{1}\boxed{0}\boxed{1}\boxed{0}$ gives 0, then (1) and (0, 1, 2), and so (0, 1, 0, 1, 3) (`pv.py brackets 1101101`). Read the Dyck path as a plane tree, the *bracket tree*: a node for every bracket, and as its children the brackets opened directly inside it. Then the children of a node are the blocks of the word its bracket spans. So a Dyck path with $n \geq 1$ up steps is the bracket word of an admissible word of Order n exactly when its tree has one root and, at every node, the sizes of the children's subtrees are valid Orders (image_bracketWord_admSet). Only p_2 enters. A parser that reads the list once decides it (isBracketTree_iff). The same pass, keeping the words of the children instead of their sizes, decodes a bracket tree back into its word (bracketDecode, invOn_bracketDecode), so the bracket trees with n nodes number $a(n)$ (ncard_isBracketTree). Of the 208012 Dyck paths with 12 up steps, $a(12) = 2652$ are bracket words.

Sweeps

The levels of a word are those of *rational Catalan* combinatorics. Armstrong, Loehr and Warrington [1] give the lattice point (x, y) the level $by - ax$; for coprime a, b the levels along a path are distinct, as ours are because $2^j \neq 3^o$. Their *sweep map* [2] sorts the steps of a path by level. As in Thomas and Williams [37], each step sits at the level where it ends, and steps are read from the highest level down. On admissible words the final halving stays last (`pv.py sweep 110111`). That the sweep of an admissible word is admissible again is proved directly (sweep_mem_admSet): a prefix of the sweep collects the steps ending at or above some level $\ell > 1$, each run of consecutive such steps climbs from below ℓ to at least ℓ, so together they multiply to more than 1. That no two words sweep to the same one is the deep part. Integer weights with a ratio close enough to $\log_2 3$ order the lattice points of Order n like our levels (exists_weights, q_pow_lt_iff_weights); with them an admissible word without its final halving is a Dyck word in their sense (dropLast_mem_dyckSetQ), and our sweep is theirs (sweep_eq_sweepZ). They proved that the sweep is a bijection on all words with given numbers of both steps [37, Thm. 6.3], so the sweep permutes the admissible words of each Order (sweep_bijOn, with their theorem as hypothesis).

Sorting by the level at which each step *starts*, lowest first, gives another map, the *start sweep*. On every word it is the sweep of the reversed word (startSweepQ_eq_sweepQ_reverse); for all words, not only Dyck words, so the start sweep is one-to-one on the admissible words of every Order (startSweep_injOn, again with that theorem as hypothesis). It sends the admissible words of Order n to admissible words exactly at the upper record Orders $1, 3, 5, 17, 29, \ldots$ (mapsTo_startSweep_iff), and there it permutes them (startSweep_bijOn_iff, same hypothesis). At an upper record Order, a later point of an admissible word, except its end, that lies below an earlier one has a level below $f(n)$ times the earlier level, $f(n)$ being the level of the whole word (lvlQ_lt_mul_of_upperRecordQ). So the word with its part before the highest point reversed, and its part after it reversed, is admissible again, and its sweep is the start sweep of the word (peakFlip_mem_admSetQ, startSweepQ_eq_sweepQ_of_upperRecordQ). At an Order n that is not an upper record, some split $n = a + b$ has $p_2(n) = p_2(a) + p_2(b)$. Take the word of n odd steps followed by halvings: its steps that start below the level $(3/2)^b$ come first in its start sweep and multiply to $f(b) < 1$, so that start sweep is not admissible (startSweepQ_upDownQ_not_mem).

With the final halving left out and put back after the sorting, the *cut sweep*, the result is admissible at every Order (cutSweep_mem_admSet). A prefix of the cut sweep holds the steps that start below some level $\ell > 1$; these multiply to at least ℓ or to at least the level before the final halving, whichever is smaller (min_le_prod_of_lower). Both are above 1, and the final halving still takes the whole below 1. The cut sweep permutes the admissible words exactly at the lower record Orders $1, 2, 7, 12, 53, \ldots$ (cutSweep_bijOn_iff, with the same hypothesis), by the same reversal around the highest point. At every other Order n the split $n = a + b$ with $f(a)$ least has $p_2(n) + 1 = p_2(a) + p_2(b)$; then the words of a and of b odd steps followed by halvings, each without its final halving, and one halving after them, form an admissible word (append_cut_mem_admSetQ) that passes a point lower than the one before its final halving. Its steps up to that point cannot come first in any cut sweep, so it is no cut sweep (not_surjOn_cutSweepQ), and two words have the same one (not_injOn_cutSweep).

The record Orders are exactly the Orders at which the linear levels of slope $p_2(n)/n$, or $(p_2(n)-1)/n$ at the lower ones, order the lattice points up to its end like ours, the two end points of the line apart: it puts those two at the same level (gridLinear_iff_upperRecord, gridLinear_iff_lowerRecord; that these are exactly the record Orders of the block factor, upperRecord_iff_blockFactor, lowerRecord_iff_blockFactor). None of this needs the number 3: the sweeps (same hypothesis) and the record Orders, also on their walk (upperRecordQ_iff_sbWalkQ), hold for the words of every odd multiplier $q \geq 3$ (not the ratio q of Chapter 3), with the factor $q/2$ for an odd step, the admissible $qx+1$ sequences of Winkler [42]. So is Theorem 1, where for $q \geq 5$ blocks 0, the *filler zeros*, can stand among the blocks, not only at the end (admissibleQ_iff_exists_chainQ, isRecordLowQ_iff_cut), and so is the first word of each Order as a chain of first words (firstWordQ_chain, firstWordQ_blocks). Neighbouring upper record Orders differ by a lower one (lowerRecord_sub_of_upperRecord).

With the filler zeros as leaves and the odd steps as nodes, the words of every odd $q \geq 3$ are trees again: the admissible words of Order n are the trees with n nodes in which, at every node, the Orders of the children keep the level above 1 before the last child and bring it below 1 after it (word_bijOn_validTreesQ). One pass reads the tree off the word (parseTreeQ_eq_some_iff). One pass also reads it back from the bracket word, with a count of the closes still due in the current column, filler zeros included (decodeQ_bracketWordQ). With one bracket per odd step a filler zero adds nothing to the counts (bracketWordOneQ_chain), which is why 10100 and 11000 clash for $q = 5$, and two words of Order 2 clash for every odd $q \geq 5$ (not_injOn_bracketWordOneQ).

In terms of levels, a bracket closes at the first later point of lower level. Scaled by 2^L, the level of the point after j steps with o odd ones is the integer $3^o\,2^{L-j}$, a 3-smooth number (A003586 [12]). The sweep sorts the points by their place among the 3-smooth numbers, which Mintz [9] counted exactly: below 2^j3^i there are $ij + \sum_{k=1}^{i} p_2(k) + \sum_{k=1}^{j} u(k)$ of them, with $u(k)$ the number of powers of 3 below 2^k (card_smoothBelowAt; `pv.py smooth 51` prints $2^5 3^4$). Listed in increasing order, the 3-smooth numbers walk through their grid by record steps only (Figure 4). From 2^j3^i the next one is $1/f(b)$ times it, for the last upper record Order $b \leq i$ ($b = 0$ is the doubling), or $2f(a)$ times it, for the last lower record Order $a \geq 1$ with $p_2(a) - 1 \leq j$, if there is one: the smaller of the two (smooth_next, smooth_succ, smooth_succ_of_pow_three). Those of $B \geq 2$ binary digits are $2^{B-p_2(i)}\,3^i$ for the Orders $i < u(B)$; with a and b the last lower and upper record Orders below $u(B)$, they rise along $i = ka \bmod (a+b)$, $k = 0, 1, \ldots$, skipping $i \geq u(B)$ (smooth_digits, exists_mul_mod_eq_of_lastRecords, smooth_class_mul_mod_lt).

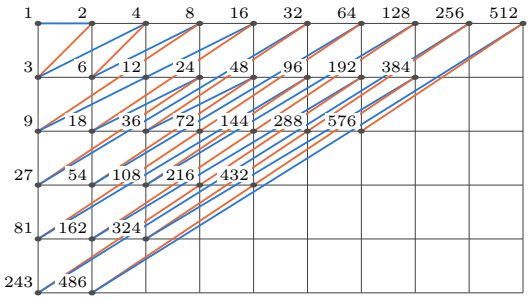


Figure 4: The 3-smooth numbers up to 576 on the grid of Mintz [9], powers of 2 to the right and powers of 3 downwards (his go up), joined in increasing order. Blue: the steps $1/f(b)$ for the upper record Orders $b = 0, 1, 3, 5$. Orange: the steps $2f(a)$ for the lower record Orders $a = 1, 2$.

Other ways to cut

Other recent work cuts the same words differently. Winkler [41] cuts them at occurrences of the local pattern 011 and counts them with a run-composition recurrence; in [42] the blocks are runs, each odd step with the halvings after it. Nakanishi [10] and Hikawa [5] remove the last 1 (Chapter 5). These cuts depend on local patterns; the cuts of Theorem 1 depend on the level, so the blocks are admissible words again and the condition on them depends on their Orders only. Cutting at record lows itself is the classical ladder factorization; as far as we know, the condition on Orders only and the counts it gives in Chapter 5 appear in none of these papers.

Key terms introduced in Chapter 4:	
record low	a level lower than all levels before it (after the leading 1).
block	a piece between two cuts: an admissible word of Order ≥ 1.
chain	$1\,A_1 \cdots A_k\,t$, the decomposition of Theorem 1.
code	the chain as a list of pairs (Order, rank), one per block.
bracket word	for each odd step, the number of brackets closing before the next odd step.
shared close	a close after the first one at the same halving.
bracket tree	the bracket word as a plane tree: a node per bracket, its children the brackets opened directly inside it.
sweep map	sort the steps of a word by the level at which they end, highest first.
start sweep	sort the steps by the level at which they start, lowest first; the *cut sweep* does so without the final halving and puts it back.
tail	the empty word or a single 0 at the end of a chain.
filler zero	for an odd multiplier $q \geq 5$: a block 0 among the other blocks of a chain.
valid Orders	$(o_1, \ldots, o_k)$ satisfying the inequality of Theorem 1.
block factor $f(o)$	$3^o/2^{p_2(o)}$, the factor by which a block of Order o changes the level.
rational Catalan	a count of lattice paths below a line of rational slope.
A003586	the 3-smooth numbers.
A206788	denominators of the semiconvergents of $\log_2 3$.

5 Counting the words

Recall that $a(n)$ is the number of admissible words of Order n (OEIS A100982), and write $m(n) = p_2(n) - 1$, the number of binary digits of 3^n minus one (A056576 [16]).

Counting with chains

By Theorem 1, a word of Order $n+1$ is a list of valid Orders with a free choice of block for each. So

$$a(n+1) \;=\; \sum_{\substack{(o_1,\dots,o_k)\ \text{valid}\\ o_1+\cdots+o_k=n}} a(o_1)\,a(o_2)\cdots a(o_k) \qquad \text{(card_admSet_succ)}.$$

For $n = 4$ the valid *compositions* are $(1,3)$, $(3,1)$ and (4), giving $a(5) = 1\cdot 2 + 2\cdot 1 + 3 = 7$.

Counting with first passages

Take any word of length $p_2(n)$ with n ones that starts with $\boxed{1}$; there are $\binom{m(n)}{n-1}$ of them. Its final level is below 1, so somewhere it passes below 1 for the first time. Cutting there gives an admissible word of some Order $i \le n$, followed by an arbitrary rest. Counting both sides:

$$\binom{m(n)}{n-1} \;=\; \sum_{i=1}^{n} a(i)\binom{m(n)-m(i)}{n-i} \qquad \text{(choose_eq_sum_admSet)}.$$

For $n = 4$: $\binom{6}{3} = 20 = 1\cdot 10 + 1\cdot 3 + 2\cdot 2 + 3\cdot 1$ (Figure 5).

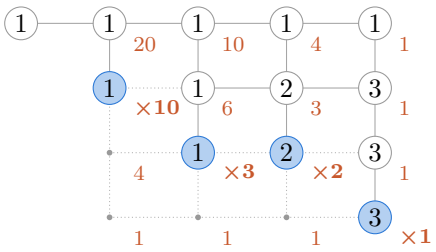


Figure 5: The first-passage identity for $n = 4$ on the counting grid of Paper 1 (Figure 1): a step right for every $\boxed{1}$, a step down for every $\boxed{0}$. As there, a white circle holds the number of walks that reach it with the level above 1 all the way, and a blue circle is the first point below level 1 of one Order, reached by $a(1),\dots,a(4) = 1,1,2,3$ walks. The small numbers count all walks from a point to the end corner, also along the dotted lines: Pascal's triangle, read from that corner. Each blue circle times its small number, added up, gives the 20 below the first $\boxed{1}$: $1\cdot 10 + 1\cdot 3 + 2\cdot 2 + 3\cdot 1$.

The same argument works for other lengths: for every $M \ge m(n-1)$,

$$\binom{M}{n-1} \;=\; a(n) + \sum_{i=1}^{n-1} a(i)\binom{M-m(i)}{n-i} \qquad \text{(choose_eq_sum_admSet_of_le)}.$$

$M = m(n)$ is the identity above; $M = m(n-1)$ gives a recurrence for $a(n)$. For $n = 5$ and $M = 6$: $a(5) = \binom{6}{4} - (5+1+2+0) = 15 - 8 = 7$. The Lean proof is an induction on n with Pascal's rule in M, anchored at $M = m(n)$.

Counting active words

Remove from an admissible word of Order $n+1$ its last 1 and the 0s after it. What remains is an active word with n ones (Chapter 2), and every active word arises exactly once. So $a(n+1)$ also counts the active words with n ones (card_persSet; cf. Hikawa and Nakanishi [6], Nakanishi [10], Hikawa [5]). This is the count computed by K. Hikawa's Python program linked from OEIS A100982. Figure 6 puts the three counts side by side for Order 5.

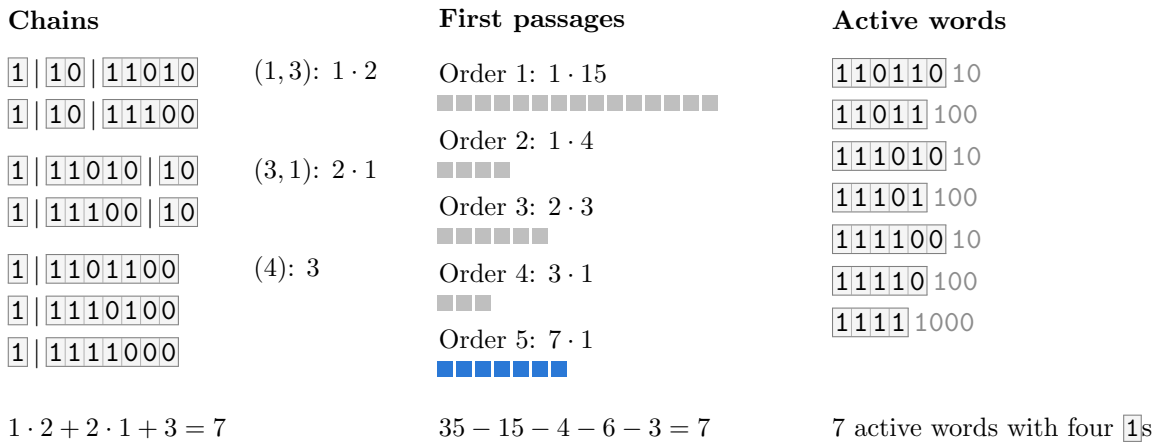


Figure 6: Three ways to count the $a(5) = 7$ admissible words of Order 5. Left: their chains, grouped by the Orders of the blocks. Middle: the $\binom{7}{4} = 35$ words of length 8 with five 1s that start with 1, one square each, by the Order of their first passage; the last row is the admissible words themselves. Right: the active words with four 1s, each followed in grey by the 1 and the 0s that were removed.

How big is $a(n)$?

Winkler [42] proved $\frac{1}{n}\binom{m(n)-1}{n-1} \le a(n) \le \frac{1}{n}\binom{m(n)}{n-1}$, with equality in the upper bound exactly at the upper record Orders of Chapter 4, and in the lower bound at the lower record Orders; there $a(n)$ is a rational Catalan number. For example $a(5) = 7 = \frac{1}{5}\binom{7}{4}$.

How often

Of the positive integers, the fraction $a(n)/2^{p_2(n)}$ lies in a class of Order n: 1/4 for Order 1, 1/16 for Order 2, 2/32 for Order 3. If the width identity of Chapter 7 holds, this is the fraction whose dropping time is $p_2(n)$, the pattern recorded in A186008 [22].

Key terms introduced in Chapter 5:	
$\boldsymbol{m(n)}$	$p_2(n) - 1$.
composition	a list of positive integers with a given sum.
A056576	$m(n)$.
A186008	class ids by dropping time, as offsets.

6 The fast map and sorting

A run of odd steps can be done in one go, by the *fast map* R. If x is odd and ends in v binary 1s, the next v steps are all odd, and with the halving that follows them they give

$$R(x) = \frac{\frac{3^v}{2^v}(x+1)-1}{2} \qquad (x \text{ odd}), \qquad R(x) = \frac{x}{2^w} \qquad (x \text{ even},\ w \text{ trailing } 0\text{s}).$$

In binary this is a stack operation: R removes the block $0\,1^v$ from the low end of x, multiplies what remains by 3^v, and adds the base-3 number $11\cdots1_3$ (v digits). For example $27 = 11011_2$ ends in two 1s: $R(27) = (9\cdot 28/4-1)/2 = 31$. After a run of v odd steps from x, the number of halvings that follow is A375782(x) [30]. Vaillant and Delarue's map [38], no ride of T ($13 \mapsto 3$), works on the index k of the odd number $2k+1$ (A324245 [26]); its change of k is A349414 [27], with one simple formula per residue of k modulo 4. And for fun: the same split of x at its rightmost 0-bit, into a head and a tail of 1s, drives the self-inverse permutation A387325 [32], which swaps the head with the length of the tail. The split also gives a continued fraction. With the point 1 for the number 0, a digit 1 below a number takes its point y to $y+1$, a digit 0 to $1/(y+1)$; so the point of x is $v+1/(y+1)$, with y the point of what remains above the 0 (markerPoint_ht; $v+1$ if nothing remains): $2+1/(3+1)$ for 27. Each further run of v 1s, with the 0 below it, is a quotient $v+1$ (the highest: $v+2$): the 1-bit markers of the trees of Hanna, Czyz and Self and of Yu-Ting [35], with the digits complemented. Read from the highest digit, the same two maps give the runs in the reverse order: the highest run first, as v, the lowest last, as $v+2$ (hiPoint_runs); and Ryde's two trees carry the two readings at the complemented node (aytPoint_complement, hcsPoint_complement).

Marked vectors

The R-trajectory of x up to its drop is recorded as a *marked vector*: an entry v for each run of v odd steps, and an entry $-h$ for each run of h halvings that R takes from an even value. (The one halving that ends a run of odd steps is part of that run and is not counted.) The last run of halvings stops as soon as the value is below the start. For example 1087 gives $[6,-2,1,-1]$ and 1855 gives $[6,-1,1,-2]$: the same odd runs, with different halvings between them (the PARI/GP code of Appendix B prints them with `t(x, 1)`). No entry is 0, and no two negative entries are neighbours.

Read back letter by letter, v 1s and a 0 for an entry v and h 0s for an entry $-h$, the marked vector of a start that drops is its word (decodeVector_eq_tBits); so its positive entries add up to the Order (order_preservation). For class ids of one Order above their cycle point the marked vector determines the class id (classId_eq_of_markedVector_eq).

Everything sorts alike

Within one Order, sort the class ids (above their cycle points) by their parity words, or by their marked vectors: the result is the same order (tBits_lt_iff_markedVector_lt). The classes can also be generated in this order directly, by a tree that extends each word by its possible next pieces (lex_enumeration; in PARI/GP, Appendix B); the rank of Chapter 2 is the position in this order.

A number sorts the same way. Read a word as a path down the Stern–Brocot tree from $1/1$, 1 to the right and 0 to the left [35]: its runs (an entry v of the marked vector, then $1+h$ 0s) are the quotients of the continued fraction of the point it reaches, the last run counted with 1 more (sbPoint_runs). So 1087, with runs $6, 3, 1, 2$, reaches $6 + 1/(3 + 1/(1 + 1/3))$. No admissible word is a prefix of another, so all of them are sorted like their points (sbPoint_lt_iff_lt). The point of a word is the mediant of the two fractions of its node (sbPoint_eq_sbMed); the intervals between those two fractions are disjoint and sorted the same way, and two of them touch exactly when one word is v followed by 0 and the other v followed by 1 and only 0s (admissible_touch_iff). Every fraction strictly inside the interval of a word is the point of exactly one word that continues it, so the classes below a word meet the fractions of its interval one to one (existsUnique_class_point); once such a word drops, the fraction lies inside the interval of its drop word (exists_admissible_between, sbNode_inside).

Key terms introduced in Chapter 6:	
$\boldsymbol{R}$	the fast map: a whole run of odd steps, or all halvings, at once.
marked vector	R-trajectory up to the drop: v per odd run, $-h$ per run of h halvings.
A375782, A349414	halvings after an odd run; index change under A324245.
A324245	the map of Vaillant and Delarue on the index k of $2k+1$.
A387325	head/tail-length swap around the rightmost 0-bit.

7 What is still open

The question "does every $x > 1$ drop?" splits into two precise parts. Recall that the level of x's parity word is a product of 3/2s and 1/2s; the *coefficient stopping time* of x is the length of the first prefix of its parity word whose level is below 1, if there is one.

- *Covering.* Every $x > 1$ has a coefficient stopping time, that is, some prefix of its parity word is admissible (Lean: Covering).
- *Width identity* (Terras [36], Conjecture 2.9). Every $x > 1$ drops exactly at its coefficient stopping time (WidthIdentity). Equivalently: every class id above 1 lies above its cycle point of Chapter 3 (ClassDrops, widthIdentity_iff_classDrops). Appendix C says why it is hard.

If every $x > 1$ drops, every $x > 1$ has a coefficient stopping time (covering_of_drops), and the two parts together give the drop for every $x > 1$ (drops_of_covering_width; for the fast map, drop_of_covering_width), hence, by Chapter 1, the classic conjecture.

The width identity would settle another question on the way: it leaves no room for a cycle. On a cycle the smallest value y comes back to itself, so the level of its parity word falls below 1 within one round, while no value on the cycle is below y. That is a failure of the width identity. So if the width identity holds, the only cycle of T is $1 \to 2 \to 1$ (no_cycle_of_widthIdentity). No other cycle is known, but proving that there is none is itself an open problem.

The drop as termination

The drop question is a termination question, and nothing in it looks past the drop. Fix a start s and allow a step $x \mapsto f(x)$ only while $x \geq s$: a value below s is final. This rewriting from s terminates exactly when s drops (DescendToMinimum.acc_iff_drops, for any map f). So the drop conjecture says: from every start $s > 1$, the rewriting with the fast map R terminates (dropConjecture_iff_acc). A proof along this line needs a measure that goes down at every allowed step, the rising ones included; none is known.

Evidence

All class ids of Orders 2 to 15 lie above their cycle points, with ratio $q \leq 251/507$ (`pv.py maxq 2 ... 15`); among the class ids below 20 000 000 the tightest case is $x = 47$ at Order 34. A direct check of every class of Orders 2 to 30 (a C++ program) confirms that all class ids drop, and that in each of these Orders the smallest difference between class id and drop value occurs at the smallest class id. The fraction of positive integers not yet covered by admissible classes of Order at most n is the fraction of active words among all words of length $p_2(n)$, that is A076227$(p_2(n))/2^{p_2(n)}$: $1/2, 1/4, 3/16, 1/8, 13/128, 19/256, 1/16, \ldots$ for $n = 0, 1, 2, \ldots$. By Terras [36] it shrinks towards 0; since all but finitely many members of a class drop, almost all x drop. Where the width identity holds, the smallest $x > 1$ not yet covered is $2, 3, 7, 27, 703, 10087, 35655, \ldots$ (OEIS A060412 [17]): each has a larger Order than every smaller start, and gives way to the next at its own Order, $0, 2, 4, 37, 51, 66, 85, \ldots$

A finite check for the width identity

The number ρ of a word cannot be large. An odd step with k letters before it adds $3^{n-o}\,2^k$ to ρ, where n is the Order and o counts the odd steps up to and including position k. Before that step the level of an admissible word is at least 1, so $2^k \le 3^{o-1}$, and each odd step adds at most 3^{n-1}. Hence $\rho \le n\,3^{n-1}$ for every admissible word of Order n (rho_le_of_admissible). The largest ρ comes from the first word alone (rho_eq_rhoMax_iff); twice what it leaves of $n\,3^{n-1}$ is the sum of the terms $3^{n-1-m}(2\cdot 3^m - 2^{p_2(m)})$ over $m < n$ (rhoMax_slack), and the term of m is below every earlier one from 1 on exactly at the lower record Orders (lowerRecord_iff_slackTerm). From below, $\rho \ge 3^n - 2^n$ for every word of Order n, with equality exactly for the words that start with all their odd steps (three_pow_le_rho_add, rho_add_eq_three_pow_iff). By the drop test of Chapter 3, every member x with $n\,3^{n-1} < d_{23}\,x$ drops at the end of its word (drop_of_large). As p_2 depends only on the Order, so does d_{23}, and the width identity only needs checking at class ids up to

$$\tau(n) = \frac{n\,3^{n-1}}{d_{23}(n)}$$

(classDrops_iff_small). This bound is small unless $2^{p_2(n)}$ comes very close to 3^n. It is above its value at every earlier Order exactly at the upper record Orders of the block factor of Chapter 4, and so is $\lfloor \tau(n) \rfloor$ (upperRecord_iff_tau, upperRecord_iff_widthBound). The reason: let n be no upper record Order, a the last upper record Order below it and $m = n - a$. Then $p_2(n) = p_2(a) + p_2(m)$, and $\tau(m) \le \tau(a)$, by induction, gives $\tau(n) < \tau(a)$ (tau_add_lt). There are 37 of these Orders up to $2 \cdot 10^6$; some of them are in Table 6, beside the walk that finds them (Figure 7).

n	$\lfloor \tau(n) \rfloor$
17	144
41	1 185
306	99 729
15 601	285 814 986
47 468	1 447 674 321
79 335	7 216 089 270
190 537	984 572 779 224

Table 6: Some records of $\lfloor \tau(n) \rfloor$.

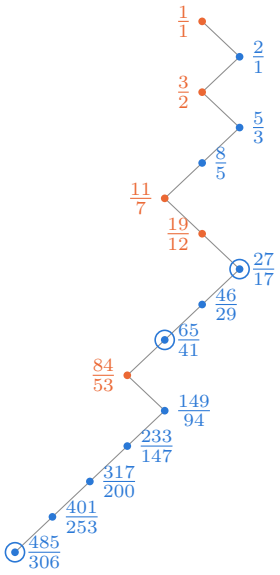


Figure 7: The walk down the Stern–Brocot tree towards $\log_2 3$ (Chapter 4); blue fractions lie above $\log_2 3$, orange ones below, and every blue denominator is a record Order of τ; those of Table 6 are ringed.

The last of them is 190 537 and the next 10 781 274 (by the walk, computed by Lean's kernel), so $\lfloor \tau(n) \rfloor$ is largest at 190 537 for $n < 10\,781\,274$ (widthBound_le_of_upperRecord). A C++ program, also outside Lean, checked that every x with $1 < x \le 984\,572\,779\,224$ drops at its coefficient stopping time; the largest Order met is 347 (at $x = 898\,696\,369\,947$).

So the width identity holds for every admissible word of Order at most 10 781 273: a computer-assisted result, since the range check enters the Lean proof as its only hypothesis (classDropsAt_le_10781273). An independent residue-class sieve and a second run of the program on other hardware reproduced the range check. The step to the Orders is formal too: if every x with $1 < x \le X$ drops at its coefficient stopping time, the width identity holds at every Order n with $\lfloor \tau(n) \rfloor \le X$ (classDropsAt_of_verified); with the records of τ, at every Order up to n as soon as X reaches $\lfloor \tau \rfloor$ of the last upper record Order up to n (classDropsAt_of_verified_record).

How large can $\tau(n)$ get? Rhin [34, Proposition, p. 160, (7)] proved that $|u_0 + u_1 \log 2 + u_2 \log 3| \ge H^{-13.3}$ for integers u_0, u_1, u_2 with $H = \max(|u_1|, |u_2|) \ge 2$ (natural logarithms). With $u_0 = 0$, $u_1 = p_2(n)$, $u_2 = -n$ this gives $2^{p_2(n)} - 3^n \ge 3^n\, p_2(n)^{-13.3}$, so $\tau(n) \le n\, p_2(n)^{13.3}/3$: the class ids to check grow at most polynomially with the Order. Mintz's exact count of the 3-smooth numbers uses the round-ups $p_2(k)$ of $k \log_2 3$; Rhin's bound says how small their excess can get. (Cited, not formalized: with $u_0 = 0$ it is the hypothesis RhinLog; his proof of (7) uses a computation by E. Dubois and P. Toffin.) Under the hypothesis $3^n \le K\, p_2(n)^C\, (2^{p_2(n)} - 3^n)$ for every $n \ge 1$ (RhinBound; Rhin's bound gives $K = 1$, $C = 14$: rhinBound_of_rhinLog), the width identity at Order n follows once every x with $1 < x \le K\, 2^C n^{C+1}$ whose word has Order n is checked to drop at its coefficient stopping time (classDropsAt_of_rhinBound; for all Orders together, classDrops_of_rhinBound). Exceptions at finitely many Orders only raise K (rhinBound_of_rhinBoundFrom). Lean's kernel checks that $K = 1$ and $C = 2$ suffice for every $n \le 10^{60}$ (rhinBoundUpTo_one_two); no proof uses it.

Two kinds of question

Which class x belongs to depends only on its lowest binary digits: a 2-adic question, blind to size. Whether x drops depends on its size. Inside one class the two agree, above the cycle point (Chapter 3); across all classes they are the covering question. This tension, 2-adic dynamics against an ordinary size condition, runs through the whole problem [8]. It also separates the fractions above from Terras's theorem [36] that almost all x drop: ours are exact finite fractions, not asymptotic densities.

Side results

Minimal class ids. The smallest class id of each Order is OEIS A122442 [20] (Chapter 3). Ten of the smallest class ids, those of Orders 37, 35, 34, 32, 28, 26, 25, 18, 17 and 15, lie on one way to a drop, the one of 27: 27, 31, 47, 71, 91, 103, 155, 167, 251 and 283 all wait for the same long way down.

Mersenne starts. For odd $k \ge 5$, if $2^k - 1$ and $2^{k+1} - 1$ both drop, the first has at least the Order of the second: both reach $(3^{k+1} - 1)/8$ after $k + 1$ odd steps and then share their way down, where the larger start drops no later (order_mersenne_succ_le).

Only 3 mod 4 *matters.* Every class id of Order ≥ 2 is 3 mod 4: even numbers drop in one step, and $4i + 1$ in two (Chapter 1; classId_mod4, order_trichotomy).

From 3 mod 4 *to* 3 mod 4. So take a value x that is 3 mod 4, with $x+1 = 2^v s$ and s odd. It takes v odd steps, up to $3^v s - 1$; a *G-step* goes on from there to the next value that is 3 mod 4, or below the start (gStep_spec, gStep_formula). The first value below the start is its drop, and a start drops exactly when its chain of G-steps gets there (gChain_drop, gChain_drop_iff). The numbers of G-steps to the drops of $3, 7, 11, 15, \ldots$ are A400742 [33]; for 27 there are 9.

Looking backwards. For $m > 1$ not divisible by 3, some odd number above m passes through m, and so drops. With $3x + 1$ and the halving after it counted as two steps, the smallest one is A350082(m) [28]; T skips the values $3x+1$, so under T it can be larger (21, not 17, for $m = 16$).

Key terms introduced in Chapter 7:	
coefficient stopping time	length of the first prefix of x's parity word with level below 1.
Covering	every $x > 1$ has a coefficient stopping time.
Width identity	every $x > 1$ drops at its coefficient stopping time (Terras 1976, Conj. 2.9).
$\boldsymbol{\tau(n)}$	$n\,3^{n-1}/d_{23}$; class ids above it drop at the end of their word.
A060412	smallest integers not yet covered; each has a larger Order than every smaller start.
A350082	smallest larger odd number passing through m, with $3x + 1$ as a step of its own.
A400742	G-steps to the drop of $4n + 3$.

A The Lean formalization

The Lean 4 project AcceleratedCollatz (Mathlib) proves every result that this paper cites with a Lean name, without `sorry`; open statements and theorems taken from the literature enter as named hypotheses. A self-contained file `ChainDecomposition.lean` holds the admissible words, the chain decomposition with its budget and record lows, the count by chains and the two binomial identities, with Mathlib as its only dependency. A second one, `SweepQ.lean`, holds the sweep, the start sweep and the cut sweep for the words of every odd $q \geq 3$. A third, `ChainQ.lean`, holds their chain decomposition with its codes, count and budget, the gaps of the record Orders and the first word of each Order. A fourth, `SmoothPlace.lean`, holds the place of $p^a q^b$ among the numbers $p^i q^j$ for any two bases. A fifth, `SmoothRuler.lean`, also with Mathlib only, is built with the project through TRulerRecord; it holds an open statement on neighbouring 3-smooth numbers that this paper does not use.

How this was made

The Lean files, the scripts and this text were written in conversation with Claude (Anthropic); M. Winkler worked with ChatGPT (OpenAI). Ideas were tried numerically first, then stated and proved in Lean, compiled locally, and every open end was logged for the next session. Some of it went better than planned. Once the words were cut at their record lows, the blocks turned out to be admissible words again, with a condition on their Orders only; the proofs of Chapters 4 and 5 then became short and clean. An independent audit, commissioned by M. Winkler, compiled an earlier version of `ChainDecomposition.lean` and checked its proofs once more with a second checker. The same workflow is open to any reader: the Lean proofs can be checked on any computer, and `pv.py` reproduces the examples. Table 7 lists the lean-files.

lean-file	content
DescendToMinimum, DropEquivalence{,T}, DropTermination	drop conjecture $\iff$ classic conjecture; the drop as termination
Numlib	general number theory
AdmissibleLength, TStaircase	admissible words, length p_2, prefix property, step bound
TParity, TRho{Lower,Max,Slack}	the step T, parity words, ρ and its least and largest values, the terms of its slack, d_{23}
TAffine, TClass, TCorrespondence, TDrop	drop test and cycle point, classes and their marked vectors, class ids, drop formula
TMod4, TSieve	Collatz-primes; only 3 mod 4 matters
TBridge, TEncode, TFold	encodings and cores; folds and ρ; marked vectors and parity words
TBudget, TChain{,Orders}, TFirst{Greedy,Orders,Word}, TRecordLow, TSplit{,Grammar}	first-drop parse, chain decomposition, Order-only validity, budget, record lows, the first word of each Order, also as a greedy sum
TAttrs, TChain{Equiv,Parse}	chains and codes as bijections, their computable form, all attributes of an odd part
TActive, TAdmSet, TBinomial{,Range}, TCount, TPersistent	the counts of Chapters 2 and 5
TPrefixCount, TRank	counting by prefix, rank and unrank
Core, MarkedVector{,Shift}, OrderPreservation, R{Map,Shift}, TMarker{Point,PointHigh,Tree}	fast map, shifts by multiples of a large power of 2, marked vectors, the point of a number
TLex, TSternBrocot{Interval,Onto,Tree}, Tree, TVectorLex, TWordPoint	tree construction, sorting, the point of a word, also as a mediant, the intervals of the words, every positive rational reached once
TRLink	the drop under R and under T
TCovering, TCycle, TFiniteCheck, TRhin{Bound,Check,Log}, TRho{Bound,Max{Bound,Check,Record}}, TRuler{Check,Pair}, TWidth{,Bound,Record}	the open parts of Chapter 7, the finite check and the records of τ and of the largest ρ over d_{23}, the bound from Rhin, no cycles under the width identity
TBracket{,Chain,Decode,Dyck,Parse,Tree}{,Q}, TBracket{Count,Drop}, TBracket{Clash,One,WordTree}Q, TNest	first drops, brackets, bracket words, bracket trees, their decoder and count; bracket words, their chain form, trees, tree parser and decoder, one-bracket words, for every $q \geq 3$
SmoothRuler, TBlockRecord, TGridOrder, TRuler{Deficit,Record}, TSmooth{Between,Class,Next,Place,Succ}, TStartSweep, TSternBrocot{,Conv}, TSweep{,Perm,Z}	the sweep map, the start and cut sweeps, record Orders as linear grids, by the block factor, on the Stern–Brocot walk and as the semiconvergents of $\log_2 3$, the 3-smooth numbers in order, each the upper or the lower step of the one before, and by their number of binary digits; the integer sweep, Thomas–Williams as hypothesis
TBisect, TFloorSum, TRecord{Floor,Walk}, TSmooth{Program,ProgramConv,Rank}, TSternBrocotCert	the A003586 program: floor sums, bisections, the walk to the last record Orders, the n-th 3-smooth number from a convergent of $\log_2 3$, also by the Stern–Brocot walk in integers, for every n
TBudgetQ, TChain{,Equiv}Q, TCountQ, TFirst{Chain,Greedy,Orders,Word}Q, TRecordLowQ, TSplitQ, TWordQ	the chain decomposition with its codes, count and budget, record lows, the first word, for the words of every odd $q \geq 3$
TBlockRecordQ, TCutSweepQ, TGridOrderQ, TLastRecordsQ, TRecordGapQ, TSmooth{Class,Next,Place,Succ}Q, TStartSweep{,Inj,Perm}Q, TSternBrocot{,Conv}Q, TSweep{,Perm}Q	the sweeps and the record Orders, also on the Stern–Brocot walk and as semiconvergents, for every odd $q \geq 3$; the numbers $2^j q^i$ in order
TGMap, TMersenne	steps from 3 mod 4 to 3 mod 4, Mersenne starts
DeadCode	unused declarations, not imported by the library

Table 7: The lean-files: all 143, and DeadCode; braces group names as in a shell, so TWidth{,Bound} is TWidth and TWidthBound.

B PARI/GP

The tree construction lists all classes up to Order N, as in Paper 1, Appendix C; the map and trajectory code computes the R-trajectory, Order and marked vector of any start.

```
\\ Tree construction: all admissible classes up to Order N,
\\ as rows [encoding, rho, class id, p2, drop, p3]
\\ q3: the powers 3^0 .. 3^N; p2: their bit lengths; r: the rows of one Order.
\\ A row with encoding s has one child per trailing 0 of s (the root has one);
\\ child i of a row with p2 = u has rho 3 rho + 2^(u - i).
a_iter_R(N) = {
  my( t=1/3, q3=vector(N+1, i, t*=3), p2=[logint(x, 2)+1 | x<-q3]
    , r=[[0, 0, 2, 1, 1, 0]]
    );
  concat(concat([r], vector(N, n
  , my( q=q3[n+1], u=p2[n], w=p2[n+1] );
    r= concat(apply(v ->
        my( s=v[1] );
        vector(if(s, valuation(s, 2), 1), i
        , my( rho=3*v[2]+(1<<(u-i)), c=lift(Mod(-rho, 1<<w)/q) );
          [(s+(1<<(i-1)))<<(w-u), rho, c, w, (c*q+rho)>>w, n]
        )
      , r))
  )));
}

\\ Map. m=0: bare display (default, 0 for a run of halvings);
\\ m=1: marked display (-w for a run of w halvings)
{ R(x, m=0) =
  my(p=x%2, w=valuation(x+p,2));
  [ if(p, w, m, -w, 0)
  , if(p, (3^w/2^w*(x+1)-1)/2, x/2^w)
  ];
}

\\ [p2, Order], vector of the steps, trajectory of x up to its drop; drop(1) = 1
{ t(x, m=0) =
  my(h, u=0, p=List(), v=List(x), x0=x);
  until(x < x0 || x == 1
  , [h,x]=R(x,m);
    listput(~p,h); listput(~v,x)
  );
  while(x && x*2 < x0, x*=2; u++);
  if(m, #p && p[#p] < 0 && p[#p]+=u);
  p=Vec(p); v=Vec(v); v[#v]=x;
  my( p3=vecsum([e|e<-p,e>0])
  , p2=logint(3^p3,2)+1
  );
  [[p2, p3], p, v];
}

\\ Test: bare (m=0, default)
? t(27)
% =
[ [59, 37]
, [2, 5, 1, 2, 3, 4, 1, 0, 3, 2, 6, 0, 4, 0, 1, 1, 1, 0, 1, 0]
, [27, 31, 121, 91, 103, 175, 445, 334, 167, 283, 319, 1822, 911, 2308, 577, 433, 325, 244, 61, 46, 23]
]
\\ Test: marked (m=1), the marked vector of the fast-map chapter
? t(27, 1)
% =
[ [59, 37]
, [2, 5, 1, 2, 3, 4, 1, -1, 3, 2, 6, -1, 4, -2, 1, 1, 1, -2, 1, -1]
, [27, 31, 121, 91, 103, 175, 445, 334, 167, 283, 319, 1822, 911, 2308, 577, 433, 325, 244, 61, 46, 23]
]
```

C Why the width identity is hard

Follow a number $x > 1$ step by step. At every moment its value is the *idealized* value, the start times the level, plus an extra part that comes from all the +1s so far. That extra part is never negative, and it changes in a known way: a halving takes it from e to $e/2$, an odd step to $(3e + 1)/2$.

Now keep a reserve: the number of halvings it would take to bring the level below 1. An odd step adds one to the reserve or leaves it as it is; each halving spends one. When the reserve is spent, the idealized value is below the start. The width identity says that the real value is below the start at that same step, for the first time. The margin can be thin: for $x = 31$ the two halvings from 244 to 61 leave a reserve of exactly one.

Why is this hard to prove? Everything we learn from "x has not dropped yet" says that the extra part is not too *small*. To have the real value below the start when the reserve is spent, we would need to know that it is not too *large*, and that would have to come from the way the extra part itself changes. Nobody knows how to get that. This is Terras's Conjecture 2.9 [36]; Winkler writes it as $\sigma^* = \sigma$ and leaves it open [40]. The same identity would already rule out cycles (Chapter 7).

References

[1] D. Armstrong, N. A. Loehr, and G. S. Warrington, *Rational parking functions and Catalan numbers*, arXiv:1403.1845 (2014).

[2] D. Armstrong, N. A. Loehr, and G. S. Warrington, *Sweep maps: a continuous family of sorting algorithms*, Adv. Math. **284** (2015), 159–185.

[3] W. Feller, *An Introduction to Probability Theory and Its Applications*, Vol. II, 2nd ed., Wiley, 1971, Chapter XII.

[4] R. L. Graham, D. E. Knuth, O. Patashnik, *Concrete Mathematics*, Addison-Wesley, Reading, MA, 1989 (6th printing, 1990).

[5] K. Hikawa, *Finite parity-vector structures in the accelerated Collatz map*, Kanazawa Gakuin Univ., ResearchGate preprint, 2026, https://doi.org/10.13140/RG.2.2.24486.28480.

[6] K. Hikawa and K. Nakanishi, *Parity vector analysis in the study of the Collatz conjecture*, ResearchGate preprint, January 2026, https://doi.org/10.13140/RG.2.2.12065.06240.

[7] T. Laarhoven and B. M. M. de Weger, *The Collatz conjecture and De Bruijn graphs*, Indag. Math. **24** (2013), 971–983; arXiv:1209.3495.

[8] J. C. Lagarias, *The* $3x+1$ *problem and its generalizations*, Amer. Math. Monthly **92** (1985), 3–23.

[9] D. J. Mintz, *2,3 sequence as binary mixture*, Fibonacci Quart. **19** (1981), 351–360.

[10] K. Nakanishi, *A structural study of parity vectors in the Collatz conjecture*, Jxiv preprint, February 2026, https://doi.org/10.51094/jxiv.3096.

[11] OEIS Foundation Inc., *Sequence A000265*, https://oeis.org/A000265.

[12] OEIS Foundation Inc., *Sequence A003586*, https://oeis.org/A003586.

[13] OEIS Foundation Inc., *Sequence A014682*, https://oeis.org/A014682.

[14] OEIS Foundation Inc., *Sequence A020914*, https://oeis.org/A020914.

[15] OEIS Foundation Inc., *Sequence A022921*, https://oeis.org/A022921.

[16] OEIS Foundation Inc., *Sequence A056576*, https://oeis.org/A056576.

[17] OEIS Foundation Inc., *Sequence A060412*, https://oeis.org/A060412.

[18] OEIS Foundation Inc., *Sequence A076227*, https://oeis.org/A076227.

[19] OEIS Foundation Inc., *Sequence A100982*, https://oeis.org/A100982.

[20] OEIS Foundation Inc., *Sequence A122442*, https://oeis.org/A122442.

[21] OEIS Foundation Inc., *Sequence A177789*, https://oeis.org/A177789.

[22] OEIS Foundation Inc., *Sequence A186008*, https://oeis.org/A186008.

[23] OEIS Foundation Inc., *Sequence A206788*, https://oeis.org/A206788.

[24] OEIS Foundation Inc., *Sequence A243115*, https://oeis.org/A243115.

[25] OEIS Foundation Inc., *Sequence A260591*, https://oeis.org/A260591.

[26] OEIS Foundation Inc., *Sequence A324245*, https://oeis.org/A324245.

[27] OEIS Foundation Inc., *Sequence A349414*, https://oeis.org/A349414.

[28] OEIS Foundation Inc., *Sequence A350082*, https://oeis.org/A350082.

[29] OEIS Foundation Inc., *Sequence A368514*, https://oeis.org/A368514.

[30] OEIS Foundation Inc., *Sequence A375782*, https://oeis.org/A375782.

[31] OEIS Foundation Inc., *Sequence A375961*, https://oeis.org/A375961.

[32] OEIS Foundation Inc., *Sequence A387325*, https://oeis.org/A387325.

[33] OEIS Foundation Inc., *Sequence A400742*, https://oeis.org/A400742.

[34] G. Rhin, *Approximants de Padé et mesures effectives d'irrationalité*, Séminaire de Théorie des Nombres, Paris 1985–86, Progr. Math. 71, Birkhäuser, Boston, 1987, 155–164.

[35] K. Ryde, *Trees of Rationals*, draft (2020), https://user42.tuxfamily.org/rationals/index.html.

[36] R. Terras, *A stopping time problem on the positive integers*, Acta Arith. **30** (1976), 241–252.

[37] H. Thomas and N. Williams, *Sweeping up zeta*, Selecta Math. **24** (2018), 2003–2034, doi:10.1007/s00029-018-0408-0, arXiv:1512.01483v5 [math.CO] 27 Feb 2018.

[38] N. Vaillant and P. Delarue, *The hidden face of the* $3x+1$ *problem. Part I: Intrinsic algorithm* (2019); linked from OEIS A324245.

[39] Ruud H.G. van Tol, *Collatz, but stop at the drop; Now with less noise* (2026).

[40] M. Winkler, *Deterministic structures in the coefficient stopping-time dynamics of the* $3x+1$ *problem*, arXiv:1709.03385v8 (2026).

[41] M. Winkler, *Coefficient first-passage words for the* $3x+1$ *map: a Beatty–Ferrers model and a power-of-two law*, preprint, September 2026, https://doi.org/10.13140/RG.2.2.31257.79200.

[42] M. Winkler, *Admissible* $qx+1$ *sequences, semiconvergents, and rational Catalan numbers*, arXiv:2609.22303 (2026).

Index

$a(n)$, **7**
active word, **7**
admissible word, **7**

block, **17**
block factor $f(o)$, **17**
bracket tree, **17**
bracket word, **17**

chain, **17**
class, **9**
class id, **9**
code, **17**
coefficient stopping time, **25**
Collatz-prime, **9**
composition, **19**
Covering, **25**
cut sweep, **17**
cycle point x^*, **9**

d_{23}, **9**
drop, **4**

family, **4**
filler zero, **17**
first passage, **7**

level, **7**
lexicographic order, **7**

$m(n)$, **19**
marked vector, **21**

odd part, **7**
OEIS
  A000265, 5, 7
  A003586, 16, 17
  A014682, 2, 4
  A020914, 5, 7
  A022921, 5, 7
  A056576, 18, 19
  A060412, 22, 25
  A076227, 6, 7, 22
  A100982, 3, 6, 7, 18, 19
  A122442, 8, 9, 24
  A177789, 8, 9
  A186008, 19
  A206788, 12, 17
  A243115, 8, 9
  A260591, 6, 7
  A324245, 20, 21
  A349414, 20, 21
  A350082, 25
  A368514, 6, 7
  A375782, 20, 21
  A375961, 2, 4
  A387325, 20, 21
  A400742, 25
Order, **7**

p_2, **4**
$p_2(n)$, **7**
p_3 (Order), **4**
parity word, **9**

q, **9**

R (fast map), **21**
rank, **7**
rational Catalan, **17**
record low, **17**
ρ, **9**

shared close, **17**
start sweep, **17**
step pattern, **4**
step T, **4**
sweep map, **17**

tail, **17**
$\tau(n)$, **25**

valid Orders, **17**

Width identity, **25**
word, **7**